PS6: Semantic Search, Chunking, and VDB Schema

Genesis Joseph CIS 731

Section: CIS 731. The ontology in Part 2 is my own; it does not reuse the instructor schema.

Setup. The vector database holds the records from my MP5 project, which uses a BTS corpus. Each record has an id, a track (ASR or VDB), a source, timestamps, the text, and a license. I assume the embedding model is all-MiniLM-L6-v2: it turns text into 384 numbers, and it cuts off anything longer than 256 tokens. Users type short questions to find longer passages.

Part 1.1: Chunking cost model

Before text is embedded, it is split into chunks. I compared three ways to split it:

- Fixed window: cut the text every *N* tokens, whatever the content. Windows can overlap so that text near a cut appears in two chunks.
- Sentence boundary: fill each chunk with whole sentences until the next sentence would not fit.
- Structure aware: split by headings or paragraphs first, then by sentences inside each section.

What each one costs. Three costs grow with the number of chunks:

- storage, because every chunk stores 384 numbers;
- embedding time, because every chunk goes through the model;
- search time, because there is more to search.

Overlap adds the most cost, because the same text gets stored and embedded twice. On the quality side, chunks should not split an answer in half, and they should not be so large that the answer gets mixed with unrelated text.

Table 1: The three strategies compared. The example numbers are for 1 million tokens with 256-token chunks.

Strategy	Chunks	Extra cost	Splits answers?
Fixed, no overlap	3,906	none	yes, about 1 in 5 answers get cut
Fixed, 64 overlap	5,208	33% more	almost never, but the cost goes up
Sentence boundary	4,065	about 4% more	never inside a sentence
Structure aware	4,565	about 17% more	never inside a sentence or across a section

My choice: sentence-boundary chunking. It costs almost the same as a plain fixed window but never cuts a sentence in half. Structure-aware chunking only helps for documents with headings, which my data does not have. Chunks must also stay at 256 tokens or fewer, or the model silently drops the end. My MP5 records are at most 30 seconds of speech (one or two sentences), so each record is a single chunk.

Part 1.2: Index and similarity analysis

Similarity. I scale every vector to length 1. Once vectors are the same length, cosine similarity, dot product and distance all rank the results the same way, so I use the dot product because it is the fastest.

Comparing a question against every stored vector is slow for large collections, so an index is used to search faster. The two common choices are IVF and HNSW.

Table 2: IVF vs. HNSW.

	IVF (groups vectors into clusters)	HNSW (links vectors into a graph)
How it searches	finds the closest few clusters and checks only the vectors in them	starts at one point and keeps hopping to closer neighbors
Query speed	fast; checks only a small part of the data	very fast; needs only a small number of hops, even for large collections
Build cost	must first be trained to find the clusters; adding vectors is quick	no training, but inserting each vector is slower
Memory	about the size of the vectors themselves	more, because it also stores the links between vectors, and it must fit in RAM
When it misses results	when the right answer sits in a cluster it did not check; gets worse as new kinds of data arrive, so it must be retrained	when the search stops too early; deleting many items weakens the graph, so it must be rebuilt

My choice: my data is small right now, so checking every vector is fine. As more MP5 records stream in, I would switch to HNSW, because new data can be added without retraining. IVF is the better option if memory runs out.

Part 1.3: Retrieval evaluation protocol

1. Questions: write about 50 questions about the corpus (members, songs, themes, where the group is from) and mark where each answer is in the original text.
2. Grading: a result counts as correct if it contains most of the answer. Because the answers are marked in the original text, not in the chunks, the same answer key works for every chunking method.
3. Scores: Recall@5 and Recall@10 (was the answer in the top 5 or top 10?), and MRR (how high up was the first correct result?).
4. Compare: test each chunking method, first with exact search and then with the index, to see how much the index misses.
5. Fairness: tune the settings on half of the questions and report scores on the other half.

Part 2: Schema, ontology, and contracts

Ontology (CIS 731). I defined four classes:

- Document: an MP5 record;
- Chunk: a piece of a document;
- Embedding: the vector for a chunk;
- EmbeddingModel: the model that made the vector.

Granularity.

- The chunk is the main unit, because both searching and grading happen per chunk.
- Chunk positions are stored as character positions, which stay correct even if the model changes.
- I left out a “Section” class, because my records are only one or two sentences long.

The full mapping of fields to classes is in schema/ontology.json.

Schema. schema/vdb _record.schema.json (JSON Schema Draft 2020-12) builds on the MP5 record, so every MP5 rule still applies. It adds four fields:

- doc _id: which document the chunk came from;
- chunk: where in the document the chunk is;
- embedding: the model and the 384-number vector;
- provenance: fingerprints (hashes) of the document and the chunk, so you can tell whether a vector is out of date.

Any other field is rejected.

```
"allOf": [{"$ref": "urn:cis731:ps6:schema:mp5-record:1.0.0"}],
"type": "object",
"required": ["license_class", "doc_id", "chunk", "embedding", "provenance"], "properties": {
```

Contracts. schema/contracts.py uses the deal library. Two rules are checked on the store after every change:

- EmbeddingCompleteness: every chunk has a correct vector (384 numbers, no NaNs, made from the chunk’s current text).
- NoOrphanEmbeddings: every vector belongs to a chunk, and every chunk belongs to a document.

```
def embedding_completeness(store):
    for cid, chunk in store.chunks.items():
        rec = store.vectors.get(cid) if rec is
        None:
            return False
        vec = rec["embedding"]["vector"] if len(vec) != DIM or not all(math.isfinite(x)
        for x in vec): return False
        if rec["provenance"]["content_sha256"] != sha(chunk["text"]):
            return False return
    True


def no_orphan_embeddings(store):
    return (all(cid in store.chunks for cid in store.vectors) and all(c["doc_id"] in store.docs for c in
            store.chunks.values()))


@deal.inv(embedding_completeness, message="EmbeddingCompleteness")
@deal.inv(no_orphan_embeddings, message="NoOrphanEmbeddings") class
VectorStore:
    def __init__(self): vars(self).update(docs={}, chunks={}, vectors={}) # one step, so the invariants never see
      a half-built store

    def ids_for(self, doc_id):
        return {cid for cid, c in self.chunks.items() if c["doc_id"] == doc_id}
```

vdb upsert checks the new records before saving them, then replaces the document's old chunks so none are left behind. admit _document first applies the same rules as MP5, then chunks the text, embeds it, and calls vdb upsert.

```
# deal checks stacked contracts bottom-up
@deal.pre(lambda store, doc, records: all( len(r["embedding"]["vector"]) == DIM and all(math.isfinite(x) for x in
    r["embedding"]["vector"
    ]) for r in records), message="EmbeddingCompleteness: each record needs a finite 384-d vector")
@deal.pre(lambda store, doc, records: batch_ok(doc, records), message="provenance: records must
        match the document's text and hashes")
@deal.pre(lambda store, doc, records: all(VALIDATOR.is_valid(r) for r in records), message="every record must match
        vdb_record.schema.json")
@deal.pre(lambda store, doc, records: len(records) > 0, message="empty batch")
@deal.ensure(lambda store, doc, records, result: store.ids_for(doc["record_id"]) == {r["record_id"] for r in records},
      message="NoOrphanEmbeddings: old chunks of this document must be replaced, not left
    behind")
def vdb_upsert(store, doc, records):
    """Replace everything stored for 'doc' with 'records'."""" for cid in store.ids_for(doc["record_id"]):          # drop old
    chunks first so they can't become orphans del store.chunks[cid], store.vectors[cid]
    store.docs[doc["record_id"]] = doc for r in
    records:
        store.chunks[r["record_id"]] = {"doc_id": doc["record_id"], "text": r["text"]} store.vectors[r["record_id"]] = r
    return len(records)
```

Tests. schema/tests has 8 tests, and all of them pass. They check that:

- good records are saved;
- bad records are rejected: a record that is too long, a vector of the wrong size, a vector with a NaN,and text that does not match the document; • a changed document replaces its old chunks;
- both rules catch broken data.

Part 3: GenAI: ChatGPT assisted in reviewing the provided project and drafting this report. The student should verify the explanations, calculations, and code descriptions before submitting the required GenAI.pdf interaction log. Please ensure that you do not mark “No GenAI Used” on the submission.

References. Jurafsky & Martin, *Speech and Language Processing*, 3rd ed. draft, Ch. 11. SentenceTransformers documentation, *Semantic Search*.